

HCI & Computer Graphics

ASSIGNMENT NO: 2

NAME: ABDUL QAYOOM

ROLL NO: 2K24/CSE/12

SUBJECT: HCI & COMPUTER GRAPHICS

CLASS: 3RD YEAR PE

SUBMITTED TO: SIR RAJESH KUMAR

Task 1 — Display Pixel Density Calculator

```
import math
```

```
W = int(input("Enter horizontal resolution (pixels): "))
```

```
H = int(input("Enter vertical resolution (pixels): "))
```

```
D = float(input("Enter physical diagonal size (inches): "))
```

```
# Total pixels
```

```
total_pixels = W * H
```

```
# Aspect ratio
```

```
gcd = math.gcd(W, H)
```

```
aspect_w = W // gcd
```

```
aspect_h = H // gcd
```

```
# DPI/PPI
```

```
diagonal_pixels = math.sqrt(W**2 + H**2)
```

```
dpi = diagonal_pixels / D
```

```
# Density classification
```

```
if dpi < 100:
```

```
    category = "Low Density (Standard Monitor)"
```

```
elif dpi <= 200:
```

```
    category = "Medium Density (HD Display)"
```

```
else:
```

```
    category = "High Density (Retina / Mobile)"
```

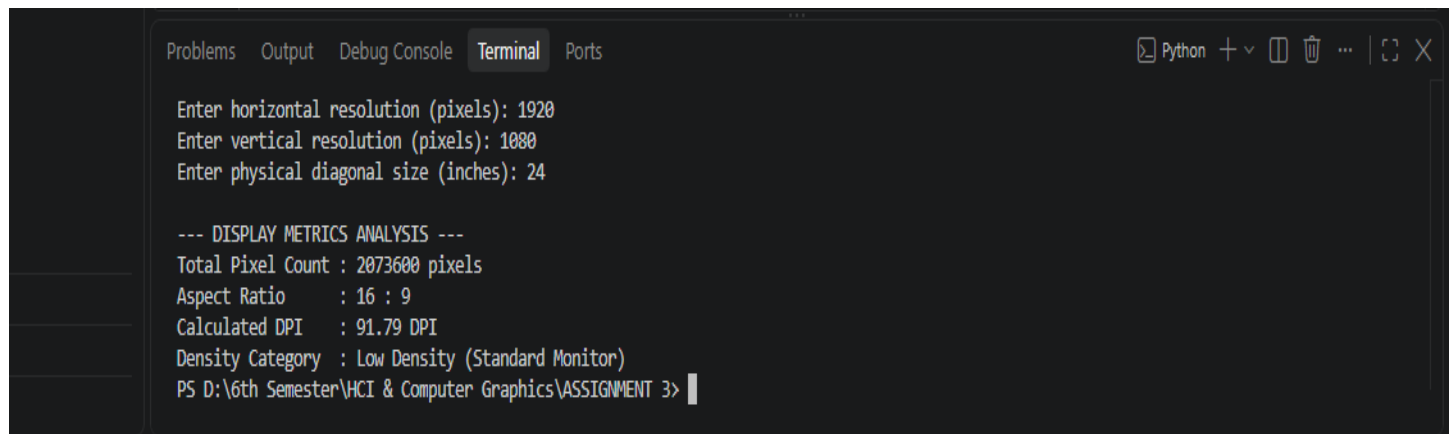
```
print("\n--- DISPLAY METRICS ANALYSIS ---")
```

```
print("Total Pixel Count :", total_pixels, "pixels")
```

```
print("Aspect Ratio    :", aspect_w, ":", aspect_h)
```

```
print("Calculated DPI   :", round(dpi, 2), "DPI")
```

```
print("Density Category :", category)
```



```
Problems Output Debug Console Terminal Ports
Enter horizontal resolution (pixels): 1920
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 24

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2073600 pixels
Aspect Ratio      : 16 : 9
Calculated DPI    : 91.79 DPI
Density Category  : Low Density (Standard Monitor)
PS D:\6th Semester\HCI & Computer Graphics\ASSIGNMENT 3>
```

Task 2 — Synthetic Image Matrix Creation

```
import numpy as np

# Create a black image

img = np.zeros((300, 400, 3), dtype=np.uint8)

# Top-left = Red

img[:150, :200] = [255, 0, 0]

# Top-right = Green

img[:150, 200:] = [0, 255, 0]

# Bottom-left = Blue

img[150:, :200] = [0, 0, 255]

# Bottom-right = White

img[150:, 200:] = [255, 255, 255]

print("--- SYNTHETIC MATRIX METRICS ---")

print("Array Shape:", img.shape)

print("Data Type:", img.dtype)

print("Total Elements:", img.size)

print("Memory Footprint:", img.nbytes, "bytes")
```



The screenshot shows a VS Code terminal window with the following content:

```
PS D:\6th Semester\HCI & Computer Graphics\ASSIGNMENT 3> & "C:\Users\ABDUL QAYOUM\AppData\Local\Programs\Python\Python314\python.exe" "d:/6th Semester/HCI & Computer Graphics/ASSIGNMENT 3/ASS2_TASK_2.py"
--- SYNTHETIC MATRIX METRICS ---
Array Shape: (300, 400, 3)
Data Type: uint8
Total Elements: 360000
Memory Footprint: 360000 bytes
PS D:\6th Semester\HCI & Computer Graphics\ASSIGNMENT 3> |
```

The terminal window has tabs for Problems, Output, Debug Console, Terminal, and Ports. The status bar at the bottom indicates: Ln 22, Col 48 (508 selected) Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit) Go Live.

Task 3 — Channel Slicing & Isolation

```
import numpy as np

from PIL import Image

import matplotlib.pyplot as plt

# 1. Load image into a NumPy array

img = np.array(Image.open("sample.jpg").convert("RGB"))

print("--- CHANNEL EXTRACTION SUMMARY ---")

print(f"Original Image Shape : {img.shape}")

# 2. Extract 2D intensity grids (Axis 2 slicing:[:, :, channel_index])

red_channel = img[:, :, 0]

green_channel = img[:, :, 1]

blue_channel = img[:, :, 2]

print(f"Red Channel 2D Shape : {red_channel.shape} | Mean Intensity: {red_channel.mean():.2f}")

print(f"Green Channel 2D Shape: {green_channel.shape} | Mean Intensity: {green_channel.mean():.2f}")

print(f"Blue Channel 2D Shape : {blue_channel.shape} | Mean Intensity: {blue_channel.mean():.2f}")


# 3. Build 3D color-isolated arrays (other two channels zeroed out)

red_only = np.zeros_like(img)

red_only[:, :, 0] = red_channel

green_only = np.zeros_like(img)

green_only[:, :, 1] = green_channel

blue_only = np.zeros_like(img)
```

```
blue_only[:, :, 2] = blue_channel
```

```
# 4. Display in a 2x3 subplot grid
```

```
fig, axes = plt.subplots(2, 3, figsize=(12, 8))
```

```
# Top row: color-isolated versions
```

```
axes[0, 0].imshow(red_only); axes[0, 0].set_title("Red Only"); axes[0, 0].axis("off")
```

```
axes[0, 1].imshow(green_only); axes[0, 1].set_title("Green Only"); axes[0, 1].axis("off")
```

```
axes[0, 2].imshow(blue_only); axes[0, 2].set_title("Blue Only"); axes[0, 2].axis("off")
```

```
# Bottom row: grayscale intensity maps
```

```
axes[1, 0].imshow(red_channel, cmap="gray"); axes[1, 0].set_title("Red Intensity"); axes[1, 0].axis("off")
```

```
axes[1, 1].imshow(green_channel, cmap="gray"); axes[1, 1].set_title("Green Intensity");  
axes[1, 1].axis("off")
```

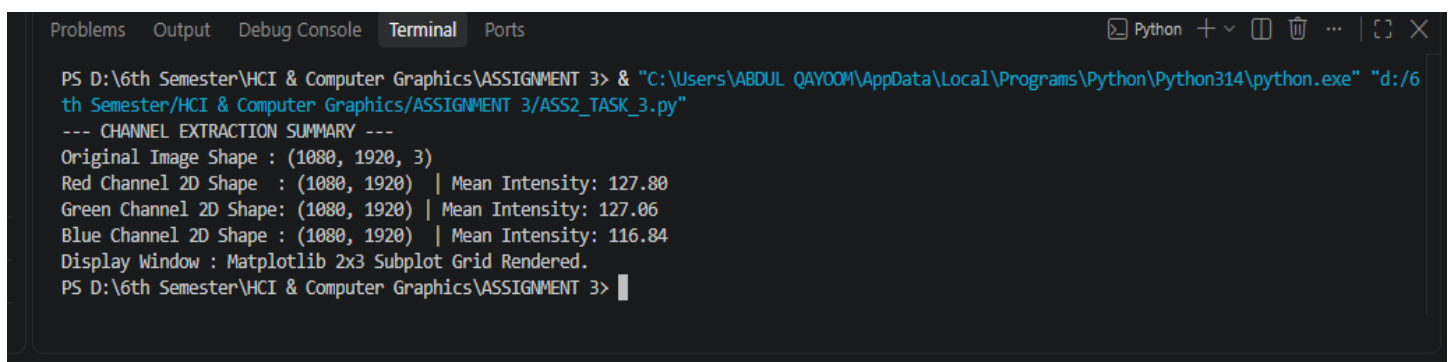
```
axes[1, 2].imshow(blue_channel, cmap="gray"); axes[1, 2].set_title("Blue Intensity"); axes[1, 2].axis("off")
```

```
plt.tight_layout()
```

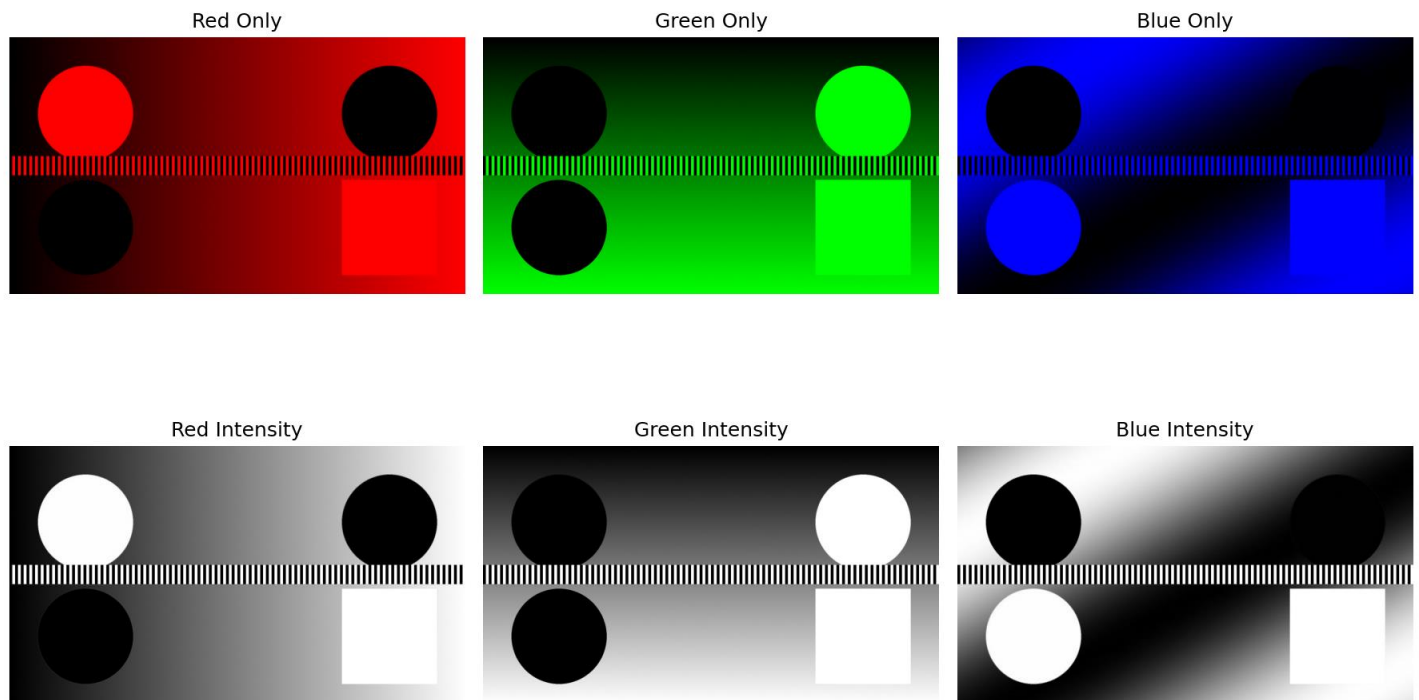
```
plt.savefig("task3_channels_output.png", dpi=150)
```

```
print("Display Window : Matplotlib 2x3 Subplot Grid Rendered.")
```

```
plt.show()
```



```
Problems Output Debug Console Terminal Ports
PS D:\6th Semester\HCI & Computer Graphics\ASSIGNMENT 3> & "C:\Users\ABDUL QAYOUM\AppData\Local\Programs\Python\Python314\python.exe" "d:/6th Semester/HCI & Computer Graphics/ASSIGNMENT 3/ASS2_TASK_3.py"
--- CHANNEL EXTRACTION SUMMARY ---
Original Image Shape : (1080, 1920, 3)
Red Channel 2D Shape : (1080, 1920) | Mean Intensity: 127.80
Green Channel 2D Shape: (1080, 1920) | Mean Intensity: 127.06
Blue Channel 2D Shape : (1080, 1920) | Mean Intensity: 116.84
Display Window : Matplotlib 2x3 Subplot Grid Rendered.
PS D:\6th Semester\HCI & Computer Graphics\ASSIGNMENT 3> |
```



Task 4 — Spatial Downsampling & Pixelation via Striding

```
import numpy as np
```

```
from PIL import Image
```

```
import matplotlib.pyplot as plt
```

```
N = 8 # step factor
```

```
# Load original image
```

```
img = np.array(Image.open("sample.jpg").convert("RGB"))
```

```
orig_shape = img.shape
```

```
orig_mem = img.nbytes
```

1. Downsample: take every N-th pixel along rows and columns

```
downsampled = img[::N, ::N, :]
```

```
down_shape = downsampled.shape
```

```
down_mem = downsampled.nbytes
```

2. Re-expand back to original size for a blocky/pixelated look

```
# np.repeat along axis 0 (rows) then axis 1 (columns)
```

```
re_expanded = np.repeat(downsampled, N, axis=0)
```

```
re_expanded = np.repeat(re_expanded, N, axis=1)
```

```
# Trim/pad in case of rounding so it matches the original exactly
```

```
re_expanded = re_expanded[: orig_shape[0], : orig_shape[1], :]
```

3. Calculate reduction percentages

```
dim_reduction_pct = (1 - (down_shape[0] * down_shape[1]) ** 0.5 /  
                      (orig_shape[0] * orig_shape[1]) ** 0.5) * 100
```

```
# Simpler equivalent, matching sample output: reduction per axis = (1 - 1/N) * 100
```

```
dim_reduction_per_axis = (1 - 1 / N) * 100
```

```
mem_reduction_pct = (1 - down_mem / orig_mem) * 100
```

```
print(f"--- DOWNSAMPLING ANALYSIS (N = {N}) ---")
```

```
print(f"Original Shape    : {orig_shape} | Memory: {orig_mem:,} bytes")
```

```
print(f"Downsampled Shape  : {down_shape} | Memory: {down_mem:,} bytes")
```

```
print(f"Re-expanded Shape  : {re_expanded.shape} | Visual: Blocky Pixelation")
```

```
print(f"Dimension Reduction: {dim_reduction_per_axis:.2f}% reduction per axis")
```

```
print(f"Memory Savings     : {mem_reduction_pct:.2f}% data reduction")
```

Optional visualization

```
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
```

```
axes[0].imshow(img); axes[0].set_title("Original"); axes[0].axis("off")
```

```
axes[1].imshow(re_expanded); axes[1].set_title(f"Pixelated (N={N})"); axes[1].axis("off")
```

```
plt.tight_layout()
```

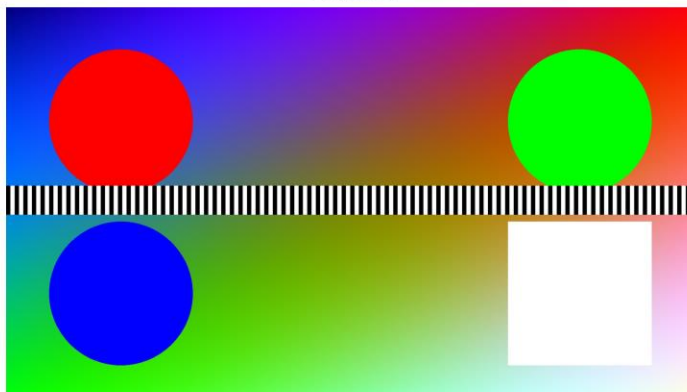
```
plt.savefig("task4_downsampling_output.png", dpi=150)
```

```
plt.show()
```

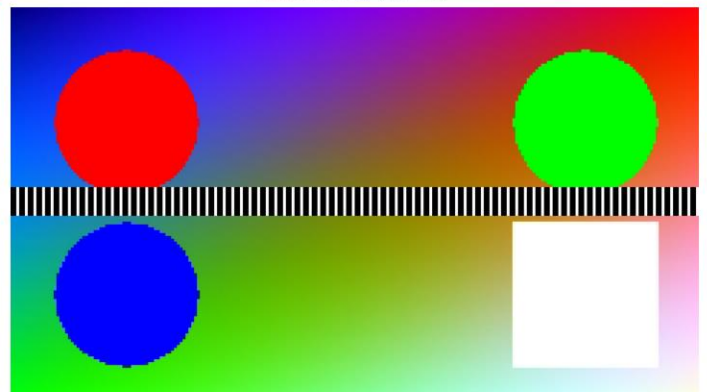
```
Problems  Output  Debug Console  Terminal  Ports
PS D:\6th Semester\HCI & Computer Graphics\ASSIGNMENT 3> & "C:\Users\ABDUL QAYOUM\AppData\Local\Programs\Python\Python314\python.exe" "d:/6th Semester/HCI & Computer Graphics/ASSIGNMENT 3/ASS2_TASK_4.py"
--- DOWNSAMPLING ANALYSIS (N = 8) ---
Original Shape      : (1080, 1920, 3) | Memory: 6,220,800 bytes
Downsampled Shape   : (135, 240, 3) | Memory: 97,200 bytes
Re-expanded Shape   : (1080, 1920, 3) | Visual: Blocky Pixelation
Dimension Reduction: 87.50% reduction per axis
Memory Savings      : 98.44% data reduction
PS D:\6th Semester\HCI & Computer Graphics\ASSIGNMENT 3> |
```

Ln 50, Col 11 (1861 selected) Spaces: 4 UTF-8 CRLF { } Python Python 3.14 (64-bit) Go Live

Original



Pixelated (N=8)



Short answer: RGB stores 3 bytes per pixel (one uint8 each for Red, Green, Blue). Adding Alpha gives you a 4th channel — also one uint8 byte per pixel — so total memory becomes:

$$\text{Memory (RGB)} = H \times W \times 3$$

$$\text{Memory (RGBA)} = H \times W \times 4$$

Ratio = $4/3 \approx 1.333...$ → +33.3% more memory

Why 4/3 specifically: you didn't make the image bigger or change any pixel's color — you just added one more same-sized plane on top of the existing three. Going from 3 planes to 4 planes is mathematically a $1/3$ increase, which is exactly 33%.

What that 4th channel actually stores: Alpha encodes per-pixel opacity/transparency — 0 = fully transparent, 255 = fully opaque, with values in between giving partial transparency (used for blending layers, cutouts, overlays, etc.). It carries no color information itself; it's purely "how much of what's behind this pixel should show through."